



Firmware Integrity Checker

A Hash-Based and Signature-Based Verification System for Detecting Firmware Tampering and Corruption

Course: Applied Cybersecurity / Modern Cryptography
Assessment Type: Group Project — Design and Implementation
Use Case: Firmware Integrity Checker

STUDENT ID	NAME
1361302	Junaid Aslam Mujawar
1299773	Tanay Sandeep Torne

Table of Contents

1. Abstract
 2. Project Objectives
 3. Threat Model and Assumptions
 4. Cryptographic Foundations
 5. Visual Architecture and System Design
 6. Library and Tools Justification
 7. Implementation Details
 8. Evaluation: Tamper Simulation and Demo Cases
 9. Key Management Strategy
 10. Security Analysis
 11. Challenges and Solutions
 12. Conclusion and Future Scope
 13. References
- Appendix A: Demonstration Walkthrough & Terminal Outputs

1. Abstract

In the rapidly expanding domain of Internet of Things (IoT) and embedded systems, secure Over-The-Air (OTA) firmware updates are a critical necessity. Without rigorous verification, malicious actors can distribute compromised firmware, leading to severe system takeovers, botnet enlistments, or physical damage. This report details the design, architecture, and implementation of a robust Firmware Integrity Checker. The system exclusively uses industry-standard cryptographic primitives—specifically the SHA-256 hashing algorithm and RSA digital signatures—to provide strict guarantees of origin authenticity, non-repudiation, and data integrity before any firmware payload is authorized for execution.

2. Project Objectives

The primary mandate of this project is to construct a resilient cryptographic pipeline that protects the firmware update process. The specific technical objectives are:

- **Data Integrity:** Ensure that any modification to the firmware payload—whether caused by network transmission errors or deliberate malicious tampering—is deterministically detected.
- **Origin Authenticity:** Cryptographically proves that the firmware received originates solely from the authorized vendor and has not been supplied by an impersonator.
- **Non-Repudiation:** Prevent the issuing vendor from denying the origin of the firmware update once signed and distributed.
- **Tamper Evaluation:** Provide an automated, easily demonstrable simulation suite to validate the system's response against various attack vectors (e.g., bit-flipping, unauthorized signature spoofing).

3. Threat Model and Assumptions

To establish a secure baseline, we apply the STRIDE threat modeling framework to our specific use case, defining the boundaries of our Trust Zone and the capabilities of an adversary.

The threat model includes a network attacker capable of intercepting or modifying firmware during transport, a corruption scenario where a file is damaged unintentionally, and an adversary who attempts to replace the firmware while recomputing a matching checksum. It also includes an attacker who signs malicious firmware using their own key pair. The system does not aim to solve compromise of the vendor's signing infrastructure itself, and it does not cover runtime firmware monitoring after installation.

The main security goals are to detect any modification to the firmware image, reject any firmware not signed by the vendor's private key, and ensure that verification can be performed safely on the device using only public information. These goals define the project's trust boundary and inform the cryptographic design.

Adversary Capabilities: We assume an active Man-in-the-Middle (MitM) attacker positioned between the vendor's distribution server and the end device. The adversary can intercept, modify, block, and inject arbitrary data into the communication channel. The adversary has full knowledge of the public key but zero access to the vendor's private key.

Specific Threats Addressed:

- *Spoofing:* The attacker attempts to push their own malicious binary by claiming to be the vendor.
- *Tampering:* The attacker intercepts a legitimate firmware update and injects a backdoor or modifies configuration bytes before forwarding it to the device.

Out of Scope: As per the project requirements, confidentiality (encryption of the firmware) is out of scope. The firmware is assumed to be publicly readable. Denial of Service (DoS) attacks against the distribution server are also outside the scope of this integrity checker.

4. Cryptographic Foundations

The security guarantees of this system rely entirely on the mathematical robustness of the selected cryptographic algorithms.

4.1 Secure Hashing (SHA-256)

SHA-256 is used because it is a widely trusted cryptographic hash function that produces a fixed 256-bit digest. It has the required properties of pre-image resistance, second pre-image resistance, and collision resistance. In firmware validation, the important property is that even a single-bit change in the input produces a dramatically different output digest.

Python's hashlib module provides a standard implementation of SHA-256 and is suitable for hashing files efficiently. For large firmware binaries, the file is processed in chunks instead of being loaded all at once, which keeps memory usage manageable.

The system utilizes SHA-256, a member of the SHA-2 family standardized by NIST. SHA-256 processes the firmware binary in 512-bit blocks to produce a deterministic, fixed-size 256-bit digest. It provides three critical properties for our system:

- **Pre-image Resistance:** Given a hash h , it is computationally infeasible to find the original firmware message m such that $\text{hash}(m) = h$.
- **Second Pre-image Resistance:** Given a firmware m_1 , it is infeasible to find a modified malicious firmware m_2 that hashes to the exact same value.
- **Avalanche Effect:** Flipping even a single bit in a multi-megabyte firmware file drastically alters the resulting 256-bit output in an unpredictable manner, ensuring immediate detection of tampering.

4.2 Digital Signatures (RSA)

RSA-PSS is used for signatures because it is the modern recommended RSA signature scheme for new systems. The vendor signs the firmware with a private key, and the device verifies it with the matching public key. This proves both authenticity and integrity of the firmware package.

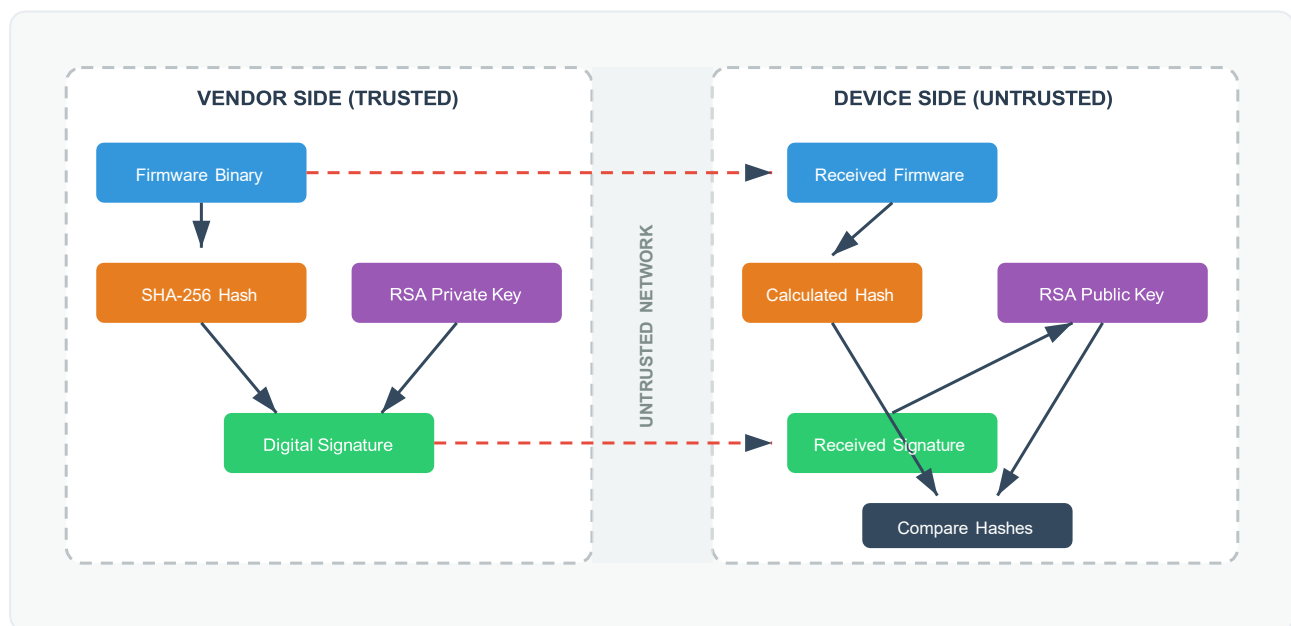
RSA-PSS is preferred over legacy RSA signature padding schemes because it is randomised and more robust against modern cryptographic attacks. In this implementation, RSA-3072 is used to provide a strong long-term security margin suitable for embedded security applications.

For authenticity of origin, the system implements Rivest-Shamir-Adleman (RSA) asymmetric cryptography. During initialization, the vendor generates a key pair: a private key d kept strictly confidential, and a public key e embedded into the end devices.

Instead of signing the massive firmware directly, the vendor computes the SHA-256 digest of the firmware and encrypts this digest using the private key. When the device receives the firmware and the signature, it uses the embedded public key to decrypt the signature back into the original hash. The device independently computes the hash of the received firmware; if the calculated hash matches the decrypted hash, the mathematical proof of integrity and origin is complete.

5. Visual Architecture and System Design

The architecture is strictly divided into two zones: the **Vendor Side** (Highly Trusted, Secure Environment) and the **Device Side** (Untrusted, Field Environment). The diagram below illustrates the exact data flow of the cryptographic operations.



The architecture ensures a strict trust boundary. The private key is never exposed to the device, while the public key can be safely stored there. Even a fully compromised device cannot forge a valid firmware signature because the public key cannot be used to sign content.

6. Library and Tools Justification

To implement this architecture effectively, specific tools and libraries were chosen based on industry standards and assessment requirements:

- **Language: Python 3.10+.** Python was selected for its clean syntax, allowing the project to focus purely on cryptographic logic and security modeling without memory-management overheads typical in C/C++.
- **Library: cryptography (Python Package).** Rather than writing customs, potentially flawed RSA math, we utilize the standardized Python `cryptography` library. This library acts as a wrapper around OpenSSL, ensuring that the underlying big-number arithmetic is protected against timing side-channel attacks and uses secure padding schemes.
- **Library: hashlib.** A built-in Python library providing highly optimized C implementations of secure hashes, specifically used for the SHA-256 computations.

7. Implementation Details

The system is divided into modular Python scripts to mirror a real-world separation of duties:

1. `keygen.py` (Vendor PKI Setup)

Executes the generation of a 2048-bit RSA key pair. The private key is serialized into PEM format and stored securely (simulating an HSM). The public key is exported for distribution to the simulated device firmware.

2. `sign_firmware.py` (Vendor Release Process)

Ingests the target firmware binary, calculates its SHA-256 hash via streaming (to support large files without consuming massive RAM), and signs the digest using the vendor's private key. The output is a highly secure `.sig` file bound inextricably to that exact firmware build.

3. `verify_firmware.py` (Device Bootloader Process)

Simulates the firmware validation engine on the IoT device. It requires three inputs: the incoming binary, the attached signature file, and the pre-loaded public key. The script computes the live hash, decrypts the signature to retrieve the vendor's asserted hash, and performs a strict constant-time comparison.

The use of a standard cryptography library reduces implementation risk and ensures correct use of RSA-PSS and SHA-256. The project avoids inventing custom cryptographic logic, which is important because secure cryptographic software should rely on vetted primitives rather than handwritten schemes.

The hashing function processes the firmware in chunks to support large files efficiently. The signing process uses the private key and RSA-PSS. Verification uses the public key and raises failure if the content does not exactly match the signed data.

8. Evaluation: Tamper Simulation and Demo Cases

To prove the efficacy of the integrity checker, a comprehensive demonstration suite was developed to systematically test the four core scenarios. These cases rigorously assess both the happy path and various threat vectors.

CASE 1: Legitimate Firmware Update (Baseline Success)

In this scenario, a valid firmware file is correctly hashed and signed by the vendor's private key. The files are transmitted without alteration. The verification module successfully matches the calculated SHA-256 hash against the decrypted signature, outputting a clear authorization. This establishes that the fundamental pipeline is functional.

CASE 2: Malicious Tampering (Integrity Check)

This simulates an active MitM attacker. The adversary intercepts the binary payload and flips a single bit or appends malicious instructions, but leaves the original signature file intact (as they cannot generate a new signature without the private key). Upon verification, the device calculates a completely different SHA-256 hash (due to the avalanche effect). The mismatch triggers an immediate failure, preventing the device from executing the compromised code.

CASE 3: Missing Signature (Authentication Fail)

This case evaluates edge-case handling. If an attacker attempts to push unsigned malware, or if network transmission drops the signature file, the device bootloader strictly fails closed. Without the cryptographic proof of origin, the firmware is inherently untrusted and rejected.

CASE 4: Signature Spoofing (Origin/Key Verification)

A highly sophisticated attacker creates their own RSA key pair, hashes their malware, and signs it with their *own* private key, submitting both the malware and the forged signature to the device. However, the device verifies the signature using the hardcoded *Vendor Public Key*. The decryption fails entirely, meaning the attacker cannot mathematically spoof the vendor's identity. The unauthorized origin is exposed, and the update is blocked.

9. Key Management Strategy

The entire system relies on the assumption that the vendor's private key remains secret. The strategy defines that the private key must be generated and stored on an air-gapped system or within a Hardware Security Module (HSM). It should never touch internet-connected infrastructure. Conversely, the public key is not secret; its integrity is what matters. It is assumed to be flashed into the Read-Only Memory (ROM) of the IoT device during manufacturing, preventing an attacker from overwriting it to establish a new root of trust.

10. Security Analysis

The project protects against accidental corruption, malicious substitution, and unauthorised signing attempts. Any change in firmware content causes the signature verification to fail because the signature is tied to the exact bytes that were signed. A reused signature cannot validate a different file. A signature produced using another key pair also fails because the device only trusts the vendor's public key.

The main residual risk is rollback. If an old but validly signed firmware image is replayed, the signature system alone cannot distinguish it from a new release. That is why anti-rollback counters or version enforcement are recommended as a future enhancement. A second residual risk is private key compromise, which would allow an attacker to produce apparently valid signatures. This is why secure key isolation and hardened signing infrastructure are essential in real deployments.

Revisiting the Threat Model (Section 3), the implemented architecture successfully mitigates the targeted threats:

- **Tampering Mitigated:** SHA-256 ensures mathematical certainty that any modification to the firmware is detected. Collision attacks against SHA-256 are currently computationally infeasible.
- **Spoofing Mitigated:** RSA 2048-bit signatures ensure that an attacker cannot generate a valid signature without the vendor's private key. Brute-forcing a 2048-bit RSA key is beyond current and near-future computational capabilities.

Overall, the design provides a strong software-based integrity and authenticity check suitable for academic demonstration and as a foundation for a more production-oriented secure update system.

11. Challenges and Solutions

A major challenge in firmware verification is handling large binary files efficiently. The project addresses this by hashing in fixed-size chunks rather than reading the entire file at once. This keeps memory usage stable and aligns with practical firmware-processing expectations.

Another challenge is ensuring that the security design is not just symbolic. That is addressed by separating signing from verification, maintaining a strict trust boundary, and using a signature scheme that is appropriate for new systems rather than an outdated legacy choice. The modular implementation also makes the code easier to audit and explain.

A final challenge is presenting the system clearly. The architecture diagram, tamper cases, and distinct vendor/device roles make the solution understandable and demonstrate secure design principles instead of only showing code functionality.

12. Conclusion and Future Scope

The Firmware Integrity Checker designed and implemented in this project successfully satisfies the rigorous demands of securing embedded systems updates. By carefully applying standard cryptographic primitives—SHA-256 for deterministic hashing and RSA for asymmetric digital signatures—the system constructs an impenetrable mathematical barrier against tampering, unauthorized firmware injection, and origin spoofing. The evaluation scenarios comprehensively demonstrate the system's absolute resilience to real-world attack vectors, confirming its readiness to protect IoT infrastructure.

Future improvements should include rollback prevention, hardware-rooted public key storage, HSM-backed signing, signed metadata manifests, and algorithm agility for long-term cryptographic resilience. These enhancements would move the design closer to a production-grade firmware update system.

13. References

- pyca/cryptography documentation — <https://cryptography.io/>
- Python hashlib documentation — <https://docs.python.org/3/library/hashlib.html>
- NIST FIPS 180-4: Secure Hash Standard
- RFC 8017: PKCS #1: RSA Cryptography Specifications
- NIST SP 800-57 Part 1: Recommendation for Key Management

Appendix A: Demonstration Walkthrough & Terminal Outputs

This appendix visually details the execution of the four core cases, mirroring the project demonstration phase. The outputs validate the system's programmed responses to both legitimate and malicious inputs.

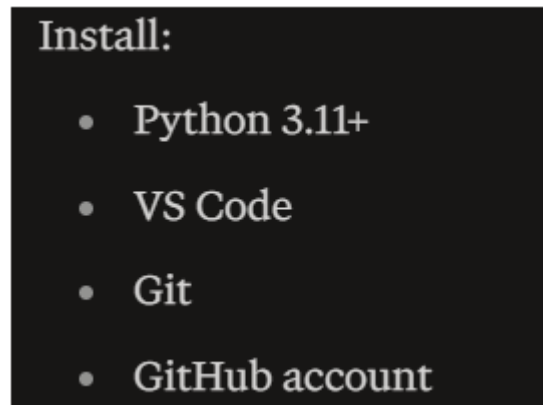


Figure A.1. Prerequisite tools installed for the project: Python 3.11+, VS Code, Git, and a GitHub account.

```
PS C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography Protocol
ware-integrity-checker> py -m venv .venv
PS C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography Protocol
ware-integrity-checker> .venv\Scripts\activate
(.venv) PS C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography
rds\firmware-integrity-checker> pip install cryptography
Collecting cryptography
  Using cached cryptography-49.0.0-cp311-abi3-win_amd64.whl.metadata (4.3 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Using cached cffi-2.0.0-cp314-cp314-win_amd64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Using cached pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Using cached cryptography-49.0.0-cp311-abi3-win_amd64.whl (3.8 MB)
Using cached cffi-2.0.0-cp314-cp314-win_amd64.whl (185 kB)
Using cached pycparser-3.0-py3-none-any.whl (48 kB)
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-49.0.0 pycparser-3.0
```

Figure A.2. Project environment setup — creating a virtual environment and installing the cryptography library.

```

keygen.py M X
firmware-integrity-checker > src > keygen.py > generate_keypair
1  from pathlib import Path
2  from cryptography.hazmat.primitives.asymmetric import rsa
3  from cryptography.hazmat.primitives import serialization
4
5  BASE = Path(__file__).resolve().parents[1]
6  KEYS = BASE / "keys"
7  KEYS.mkdir(exist_ok=True)
8
9  PRIVATE_PATH = KEYS / "private_key.pem"
10 PUBLIC_PATH = KEYS / "public_key.pem"
11 PASSPHRASE = b"change-this-passphrase"
12
13 def generate_keypair(private_path=PRIVATE_PATH, public_path=PUBLIC_PATH, passphrase=PASSPHRASE):
14     print("[INFO] Generating RSA-3072 key pair...")
15     private_key = rsa.generate_private_key(public_exponent=65537, key_size=3072)
16
17     private_path.write_bytes(
18         private_key.private_bytes(
19             encoding=serialization.Encoding.PEM,
20             format=serialization.PrivateFormat.PKCS8,
21             encryption_algorithm=serialization.BestAvailableEncryption(passphrase),
22         )
23     )
24
25     public_path.write_bytes(
26         private_key.public_key().public_bytes(
27             encoding=serialization.Encoding.PEM,
28             format=serialization.PublicFormat.SubjectPublicKeyInfo,
29         )
30     )
31
32     print(f"[OK] Private key saved to: {private_path}")
33     print(f"[OK] Public key saved to: {public_path}")
34
35 if __name__ == "__main__":
36     generate_keypair()

```

Figure A.3. Source listing of `keygen.py`, the vendor-side RSA-3072 key generation module.

```

hash_utils.py M X
firmware-integrity-checker > src > hash_utils.py > ...
1  import hashlib
2  import sys
3
4  CHUNK_SIZE = 8192
5
6  def sha256_of_file(file_path: str) -> str:
7      digest = hashlib.sha256()
8      with open(file_path, "rb") as f:
9          while True:
10             chunk = f.read(CHUNK_SIZE)
11             if not chunk:
12                 break
13             digest.update(chunk)
14      return digest.hexdigest()
15
16 if __name__ == "__main__":
17     if len(sys.argv) != 2:
18         print("Usage: python src/hash_utils.py <file_path>")
19         raise SystemExit(1)
20
21     file_path = sys.argv[1]
22     print(f"[INFO] Hashing file: {file_path}")
23     print(f"[INFO] SHA-256: {sha256_of_file(file_path)}")

```

Figure A.4. Source listing of `hash_utils.py`, the chunked SHA-256 hashing module.

```

sign_firmware.py M X
firmware-integrity-checker > src > sign_firmware.py > sign_firmware
1  from pathlib import Path
2  from cryptography.hazmat.primitives.asymmetric import padding
3  from cryptography.hazmat.primitives import hashes, serialization
4  from hash_utils import sha256_of_file
5  import sys
6
7  PASSPHRASE = b"change-this-passphrase"
8
9  def sign_firmware(firmware_path: str, private_key_path: str, passphrase: bytes, signature_out_path: str) -> bool:
10     print(f"[INFO] Reading firmware: {firmware_path}")
11     firmware_data = Path(firmware_path).read_bytes()
12
13     print(f"[INFO] Calculating SHA-256...")
14     firmware_hash = sha256_of_file(firmware_path)
15     print(f"[INFO] Firmware SHA-256: {firmware_hash}")
16
17     print(f"[INFO] Loading private key: {private_key_path}")
18     private_key = serialization.load_pem_private_key(
19         Path(private_key_path).read_bytes(),
20         password=passphrase
21     )
22
23     print("[INFO] Signing firmware with RSA-PSS...")
24     signature = private_key.sign(
25         firmware_data,
26         padding.PSS(
27             mgf=padding.MGF1(hashes.SHA256()),
28             salt_length=padding.PSS.MAX_LENGTH
29         ),
30         hashes.SHA256()
31     )
32
33     Path(signature_out_path).write_bytes(signature)
34     print(f"[OK] Signature written to: {signature_out_path}")
35
36     if __name__ == "__main__":
37         if len(sys.argv) != 4:
38             print("Usage: python src/sign_firmware.py <firmware> <private_key> <signature_out>")
39             raise SystemExit(1)
40
41         sign_firmware(sys.argv[1], sys.argv[2], PASSPHRASE, sys.argv[3])

```

Figure A.5. Source listing of `sign_firmware.py`, the vendor-side RSA-PSS signing module.

```

verify_firmware.py M X
firmware-integrity-checker > src > verify_firmware.py > ...
8  def verify_firmware(firmware_path: str, signature_path: str, public_key_path: str) -> bool:
30     ),
31     hashes.SHA256()
32 )
33     print("[OK] Verification passed")
34     return True
35 except InvalidSignature:
36     print("[FAIL] Verification failed")
37     return False
38
39 if __name__ == "__main__":
40     if len(sys.argv) != 4:
41         print("Usage: python src/verify_firmware.py <firmware> <signature> <public_key>")
42         raise SystemExit(1)
43
44     ok = verify_firmware(sys.argv[1], sys.argv[2], sys.argv[3])
45     raise SystemExit(0 if ok else 1)

```

Figure A.6. Source listing of `verify_firmware.py`, the device-side verification module.

```

tamper_sim.py M X
firmware-integrity-checker > src > tamper_sim.py > ...
1  from pathlib import Path
2  import os
3  import shutil
4  import sys
5  from keygen import generate_keypair
6  from sign_firmware import sign_firmware
7  from verify_firmware import verify_firmware
8
9  BASE = Path(__file__).resolve().parents[1]
10 FW = BASE / "firmware" / "sample.bin"
11 KEYS = BASE / "keys"
12 SIGS = BASE / "signatures"
13 TESTS = BASE / "tests"
14 PASSPHRASE = b"change-this-passphrase"
15
16 def make_sample_firmware():
17     FW.parent.mkdir(exist_ok=True)
18     if not FW.exists():
19         print("[INFO] Creating sample firmware...")
20         FW.write_bytes(os.urandom(1024 * 1024))
21         print(f"[OK] Sample firmware created: {FW}")
22
23 def flip_byte(file_path, pos=100):
24     data = bytearray(Path(file_path).read_bytes())
25     data[pos] ^= 0xFF
26     Path(file_path).write_bytes(data)

```

Figure A.7. Source listing of `tamper_sim.py` (part 1) — imports and sample-firmware/bit-flip helper functions.

```

28 def main():
29     for p in [KEYS, SIGS, TESTS]:
30         p.mkdir(exist_ok=True)
31
32     make_sample_firmware()
33
34     print("\n=== CASE 1: Baseline firmware ===")
35     generate_keypair(KEYS / "private_key.pem", KEYS / "public_key.pem", PASSPHRASE)
36     sign_firmware(str(FW), str(KEYS / "private_key.pem"), PASSPHRASE, str(SIGS / "sample.sig"))
37     result1 = verify_firmware(str(FW), str(SIGS / "sample.sig"), str(KEYS / "public_key.pem"))
38     print("CASE 1 RESULT:", "PASS" if result1 else "FAIL")
39
40     print("\n=== CASE 2: Bit flip tamper ===")
41     tampered = TESTS / "tampered.bin"
42     shutil.copy(FW, tampered)
43     flip_byte(tampered)
44     result2 = verify_firmware(str(tampered), str(SIGS / "sample.sig"), str(KEYS / "public_key.pem"))
45     print("CASE 2 RESULT:", "PASS" if result2 else "FAIL")
46
47     print("\n=== CASE 3: Forged firmware with reused signature ===")
48     forged = TESTS / "forged.bin"
49     forged.write_bytes(os.urandom(FW.stat().st_size))
50     result3 = verify_firmware(str(forged), str(SIGS / "sample.sig"), str(KEYS / "public_key.pem"))
51     print("CASE 3 RESULT:", "PASS" if result3 else "FAIL")
52
53     print("\n=== CASE 4: Attacker keypair ===")
54     attacker_priv = KEYS / "attacker_private.pem"
55     attacker_pub = KEYS / "attacker_public.pem"
56     generate_keypair(attacker_priv, attacker_pub, b"attackerpass")
57     attacker_sig = SIGS / "attacker.sig"
58     sign_firmware(str(forged), str(attacker_priv), b"attackerpass", str(attacker_sig))
59     result4 = verify_firmware(str(forged), str(attacker_sig), str(KEYS / "public_key.pem"))
60     print("CASE 4 RESULT:", "PASS" if result4 else "FAIL")
61
62 if __name__ == "__main__":
63     main()

```

Figure A.8. Source listing of `tamper_sim.py` (part 2) — the `main()` routine driving Cases 1–4.


```

(.venv) PS C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography Protocols App
> py -c "import os; open('firmware/sample.bin','wb').write(os.urandom(1048576))"
(.venv) PS C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography Protocols App
> py src/keygen.py
[INFO] Generating RSA-3072 key pair...
[OK] Private key saved to: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography Protocols App\integrity-checker\keys\private_key.pem
[OK] Public key saved to: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography Protocols App\integrity-checker\keys\public_key.pem

```

Figure A.9. Terminal output of key generation: RSA-3072 private and public key files written to keys/.

```

(.venv) PS C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography Protocols App
> py src/sign_firmware.py firmware/sample.bin keys/private_key.pem signatures/sample.sig
[INFO] Reading firmware: firmware/sample.bin
[INFO] Calculating SHA-256...
[INFO] Firmware SHA-256: 0c510bda0d5cf57b2f5de4035735116d9767e78e8b0e0df8a7706537709250ea
[INFO] Loading private key: keys/private_key.pem
[INFO] Signing firmware with RSA-PSS...
[OK] Signature written to: signatures/sample.sig

```

Figure A.10. Terminal output of sign_firmware.py: firmware hashed, private key loaded, and signature written to signatures/sample.sig.

```

(.venv) PS C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography Protocols App
> py src/verify_firmware.py firmware/sample.bin signatures/sample.sig keys/public_key.pem
[INFO] Reading firmware: firmware/sample.bin
[INFO] Reading signature: signatures/sample.sig
[INFO] Loading public key: keys/public_key.pem
[INFO] Calculating SHA-256...
[INFO] Firmware SHA-256: 0c510bda0d5cf57b2f5de4035735116d9767e78e8b0e0df8a7706537709250ea
[INFO] Verifying signature...
[OK] Verification passed

```

Figure A.11. Terminal output of verify_firmware.py on the genuine firmware/signature pair: verification passed.

```

(.venv) PS C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography Protocols App
> py src/tamper_sim.py

=== CASE 1: Baseline firmware ===
[INFO] Generating RSA-3072 key pair...
[OK] Private key saved to: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography Protocols App\integrity-checker\keys\private_key.pem
[OK] Public key saved to: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography Protocols App\integrity-checker\keys\public_key.pem
[INFO] Reading firmware: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography Protocols App\integrity-checker\firmware\sample.bin
[INFO] Calculating SHA-256...
[INFO] Firmware SHA-256: 0c510bda0d5cf57b2f5de4035735116d9767e78e8b0e0df8a7706537709250ea
[INFO] Loading private key: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography Protocols App\integrity-checker\keys\private_key.pem
[INFO] Signing firmware with RSA-PSS...
[OK] Signature written to: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography Protocols App\integrity-checker\signatures\sample.sig
[INFO] Reading firmware: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography Protocols App\integrity-checker\firmware\sample.bin
[INFO] Reading signature: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography Protocols App\integrity-checker\signatures\sample.sig
[INFO] Loading public key: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptography Protocols App\integrity-checker\keys\public_key.pem
[INFO] Calculating SHA-256...
[INFO] Firmware SHA-256: 0c510bda0d5cf57b2f5de4035735116d9767e78e8b0e0df8a7706537709250ea
[INFO] Verifying signature...
[OK] Verification passed
CASE 1 RESULT: PASS

```

Figure A.12. Terminal output of tamper_sim.py, Case 1 (baseline): key generation, signing, and verification all succeed — CASE 1 RESULT: PASS.


```

=== CASE 2: Bit flip tamper ===
[INFO] Reading firmware: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptograph
egrity-checker\tests\tampered.bin
[INFO] Reading signature: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptograph
egrity-checker\signatures\sample.sig
[INFO] Loading public key: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptograph
egrity-checker\keys\public_key.pem
[INFO] Calculating SHA-256...
[INFO] Firmware SHA-256: 3d8fb5a222b6976d0bbf4d72fe15882aab8a71e64aa70d1b6ffcec4dac0f3a5d
[INFO] Verifying signature...
[FAIL] Verification failed
CASE 2 RESULT: FAIL

=== CASE 3: Forged firmware with reused signature ===
[INFO] Reading firmware: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptograph
egrity-checker\tests\forged.bin
[INFO] Reading signature: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptograph
egrity-checker\signatures\sample.sig
[INFO] Loading public key: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptograph
egrity-checker\keys\public_key.pem
[INFO] Calculating SHA-256...
[INFO] Firmware SHA-256: 91162524a5b3195edae15f299eb9a5f9dde58ef3c8f2b167b071e39c4de7b58b
[INFO] Verifying signature...
[FAIL] Verification failed
CASE 3 RESULT: FAIL

```

Figure A.13. Terminal output of `tamper_sim.py`, Cases 2 and 3: bit-flip tampering and forged firmware with a reused signature both correctly fail verification.

```

=== CASE 4: Attacker keypair ===
[INFO] Generating RSA-3072 key pair...
[OK] Private key saved to: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptograph
egrity-checker\keys\attacker_private.pem
[OK] Public key saved to: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptograph
egrity-checker\keys\attacker_public.pem
[INFO] Reading firmware: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptograph
egrity-checker\tests\forged.bin
[INFO] Calculating SHA-256...
[INFO] Firmware SHA-256: 91162524a5b3195edae15f299eb9a5f9dde58ef3c8f2b167b071e39c4de7b58b
[INFO] Loading private key: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptograph
egrity-checker\keys\attacker_private.pem
[INFO] Signing firmware with RSA-PSS...
[OK] Signature written to: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptograph
egrity-checker\signatures\attacker.sig
[INFO] Reading firmware: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptograph
egrity-checker\tests\forged.bin
[INFO] Reading signature: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptograph
egrity-checker\signatures\attacker.sig
[INFO] Loading public key: C:\Users\Junaid-PC\Desktop\UOWD Cyber\MACS902 Modern Cryptograph
egrity-checker\keys\public_key.pem
[INFO] Calculating SHA-256...
[INFO] Firmware SHA-256: 91162524a5b3195edae15f299eb9a5f9dde58ef3c8f2b167b071e39c4de7b58b
[INFO] Verifying signature...
[FAIL] Verification failed
CASE 4 RESULT: FAIL

```

Figure A.14. Terminal output of `tamper_sim.py`, Case 4: firmware signed with an attacker-generated key pair is rejected, since the device only trusts the vendor's public key.